

■ ARCHITECTURE DECISION RECORD

AIが設計書を 歴史書にする前に

現在の仕様と判断履歴を分けるコンテキスト設計

■ ARCHITECTURE DECISION RECORD

- 大田 一希 (Kazuki Ota)
- Microsoft / Cloud Solution Architect & Evangelist
- X: **@okazuki**
- zenn: <https://zenn.dev/okazuki>



AI を使って開発していると...

過去の経緯が書かれている設計書が出来上がる。

SharePoint Indexer

SharePoint のファイルを定期的を取得し、
AI Search へ登録する。

フォルダーを順にたどり、
50 フォルダーごとに処理を区切る。

以前はすべてを一度に取得していたが、
10 分でタイムアウトしたため分割した。
その後、実行履歴も肥大化したため、
区切りごとに処理を再開する方式へ変更した。

こう書いてほしい

現状の最新の情報だけ書いておいてほしい。

SharePoint Indexer

SharePoint のファイルを定期的 to 取得し、
AI Search へ登録する。

フォルダーを順にたどり、
50 フォルダーごとに処理を区切る。

特に指示をせずにAIに書いてもらおうと...

過去の間緯まで一緒に残る

ドキュメントが読みにくくなる。

対応策

そこで、ドキュメント用のSKILLに以下のようなことを追記

ドキュメントには過去の経緯を含めず、
最新の情報だけを残してください。

ただ、コードには

過去の問題と対応がドキュメントコメントに残った

```
/// <remarks>
/// Issue #187 - 生成中セッション切替時の送信不能問題対応。
/// <para>
/// 各送信は PendingSend としてセッション ID 単位で追跡される。
/// _isSending は「Active Session に進行中の送信があるか」を表す
/// 派生状態であり、セッション切替時は再計算される。
/// </para>
/// <para>
/// セッション切替時、旧セッションのストリーミングは
/// サーバ側でキャンセルしない (stale 化方針) 。
/// バックグラウンドで完了させて履歴に永続化させ、
/// Active Session 側の UI には混入させない。
/// </para>
/// </remarks>
public partial class ChatPage(
    IChatService chatService, ISnackbar snackbar, ICurrentUser currentUser,
    IJSRuntime jsRuntime, BrowserTimeZoneProvider timeZoneProvider,
    TimeProvider timeProvider, ILogger<ChatPage> logger) { /* ... */ }
```

そこで

コード用の**SKILL**も
別に用意した

問題解決

ただ、そうすると今度は
**AIが同じ間違いを
何度もするようになった**

たとえば、この間

AzureのBicepで タイミング問題が発生

BicepはAzureリソースをIaCで書くための言語です。

一度は回避できた

AI と共に問題調査・解決！！

でも、その後もBicepを触るたび
同じ間違いをした

そのたびにAIが
エラー対応で迷走...
トークンを無駄に消費

そこで、次に開発したプログラムでは
ADRを入れてみた

ADR

Architecture Decision Record

新しいものではなく、昔からあるやり方です。

Microsoft、AWS、Googleの公式ドキュメントでも紹介されています。

アーキテクチャ決定レコード (ADR) は、ソリューション アーキテクトの最も重要な成果物の 1 つです。

引用: [アーキテクチャデザイン レコード \(ADR\) を維持する](#)

ざっくり言うと

重要な設計判断を
決まった形で残す

ADRに残すもの

- 判断したこと
- そのときの状況と理由
- 却下した代替案
- 影響やトレードオフ

AIに渡したのは 2つのSKILL

- **Design Doc Maintenance** — 「今どうなっているか」
現在の姿だけを上書きし、理由はADRへリンクする
- **ADR Workflow** — 「なぜそう決めたか」
重要な判断の背景・代替案・トレードオフを残す

ADRを書く判断基準

次のどれかに 当てはまる決定

- 全体の構造に影響 — アーキテクチャ・プロジェクト構成
- 開発全体に影響 — 開発プロセス・主要ツール
- 戻すコストが高い — データ永続化・外部連携

変数名やprivateメソッド分割など、些細な実装詳細はADR化しない。

実際のADRテンプレート

Status

Proposed / Accepted / Superseded / Deprecated

Context

背景・制約・代替案と却下理由

Decision

決定内容と適用範囲

Consequences

メリット・トレードオフ・影響

AIがADRも読むようになると

前にやめた方法を
繰り返しにくくなった

そして設計書には
ほぼ最新の状態だけが
残るようになった

実際のリポジトリを見てみると

ADRが18件

github.com/runceel/wsl-containers-desktop

- 0013 一覧を差分更新する
- 0017 ViewModelを機能単位に分割する
- 0018 表の列幅変更を採用する

たとえば、ADR-0017

Context

ContainersViewModelが約1,000行

Decision

共有ファサードを残して内部を4分割

Rejected

画面ごとにViewModelを分ける

→ 共有状態が壊れるため

設計書のほうは、こんな感じ

- > このドキュメントは現時点のスナップショットです。
- > 経緯・検討過程は書きません。

``ContainersViewModel` (`ViewModels/ContainersViewModel.cs`)` は、
``ContainersPage``・``LogsWindow``・``ShellWindow``が共有する
DIシングルトンの公開XAML/コマンドファサード
(`[ADR-0017](../adr/0017-split-containersviewmodel-and-runtime-client-into-focused-components.md)`)。

別のリポジトリでは、決定を変更

ADR-0002 BlobTriggerを採用
↓ Blob削除を検出できない
ADR-0003 Timer定期クロールへ変更

古いADRは上書きせず、**Superseded**として残します。

Pull RequestにADRがあれば

重要な設計判断が あったのかもしれない

少ししっかり見よう、という判断材料になります。

ただ、チーム開発では
**ADR同士の判断が
ぶつかるかもしれない**

全体を見ながら
マージを判断する人が必要

ADRが増えてきたら 探すためのCLIや スクリプトも欲しいかも

ここまでは、まだ試せていません。

今日のまとめ

- 設計書は、**最新の状態**だけにする
- 過去の設計判断は、**ADRに残す**
- AIは、**設計書とADR**を読んで変更する

ADR自体は昔からある

AIのおかげで 続けやすくなった

人間だけで完璧に続けるのは、なかなか大変でした。

こんなときは **ADRを試してみてください**

- AIが同じ間違いを繰り返す
- ドキュメントが歴史書になっている

■ ARCHITECTURE DECISION RECORD

ご清聴
ありがとうございました